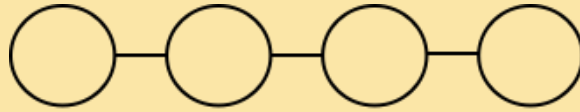




# 1. 线性表的定义和基本操作

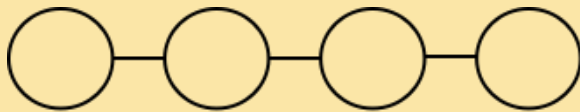


## 线性表

由 $n$  ( $n \geq 0$ ) 个数据特性相同的元素构成的有限序列



# 1. 线性表的定义和基本操作



## 线性表的特点

- (1) 线性表是一种逻辑结构
- (2) 表中元素的个数有限
- (3) 表中数据元素类型相同
- (4) 除第一个元素外，每个数据元素均只有一个前驱
- (5) 除最后一个元素外，每个数据元素均只有一个后继



# 1. 线性表的定义和基本操作

ADT List{

数据对象 :  $D=\{a_i | a_i \in \text{ElemSet}, i=1, 2, \dots, n, n \geq 0\}$

数据关系 :  $R=\{<a_{i-1}, a_i> | a_{i-1}, a_i \in D, i=2, \dots, n\}$

基本操作 :

**InitList**(&L)

操作结果 : 构造一个空的线性表L。

**DestroyList**(&L)

初始条件 : 线性表L已存在。

操作结果 : 销毁线性表L。

**ClearList**(&L)

初始条件 : 线性表L已存在。

操作结果 : 将L重置为空表。

**ListEmpty**(L)

初始条件 : 线性表L已存在。

操作结果 : 若L为空表, 则返回true, 否则返回false。

**ListLength**(L)

初始条件 : 线性表L已存在。

操作结果 : 返回L中数据元素的个数。

**GetElem**(L, i, &e)

初始条件 : 线性表L已存在, 且  $1 \leq i \leq \text{ListLength}(L)$ 。

操作结果 : 用e返回L中第i个数据元素的值。

**LocateElem**(L, e)

初始条件 : 线性表L已存在。

操作结果 : 返回 L 中第 1 个值与 e 相同的元素在 L 中的位置。若这样的数据元素不存在, 则返回值为 0。

**PriorElem**(L, cur\_e, &pre\_e)

初始条件 : 线性表L已存在。

操作结果 : 若cur\_e是L的数据元素, 且不是第一个, 则用pre\_e返回其前驱, 否则操作失败, pre\_e 无定义。

**NextElem**(L, cur\_e, &next\_e)

初始条件 : 线性表L已存在。

操作结果 : 若cur\_e是L的数据元素, 且不是最后一个, 则用next\_e返回其后继, 否则操作失败, next\_e 无定义。

**ListInsert**(&L, i, e)

初始条件 : 线性表L已存在, 且  $1 \leq i \leq \text{ListLength}(L) + 1$ 。

操作结果 : 在L中第i个位置之前插入新的数据元素e, L的长度加1。

**ListDelete**(&L, i)

初始条件 : 线性表L已存在且非空, 且  $1 \leq i \leq \text{ListLength}(L)$ 。

操作结果 : 删除L的第i个数据元素, L的长度减1。

**TraverseList**(L)

初始条件 : 线性表L已存在。

操作结果 : 对线性表L进行遍历, 在遍历过程中对L的每个节点访问一次。

}ADT List

**线性表** 由n ( $n \geq 0$ ) 个数据特性相同的元素构成的有限序列



# 1. 线性表的定义和基本操作

## 线性表的基本操作

|                                     |                                |
|-------------------------------------|--------------------------------|
| <b>InitList (&amp;L)</b>            | 初始化表，构造一个空线性表                  |
| <b>Length (L)</b>                   | 求表长，返回线性表L的长度，即L中数据元素的个数       |
| <b>LocateElem (L,e)</b>             | 按值查找操作，在表L中查找具有给定关键字值的元素       |
| <b>GetElem (L,i)</b>                | 按位查找操作，获取表L中第i个位置的元素的值         |
| <b>ListInsert (&amp;L,i,e)</b>      | 插入操作，在表L中的第i个位置上插入指定元素e        |
| <b>PrintList (L)</b>                | 输出操作，按前后顺序输出线性表L的所有元素值         |
| <b>ListDelete (&amp;L,i,&amp;e)</b> | 删除操作，删除表L中第i个位置的元素，并用e返回删除元素的值 |
| <b>Empty (L)</b>                    | 判空操作，若L为空表，则返回true，否则返回false   |
| <b>DestroyList (&amp;L)</b>         | 销毁操作，销毁线性表，并释放线性表L所占用的内存空间     |

**线性表** 由n ( $n \geq 0$ ) 个数据特性相同的元素构成的有限序列



# 2. 线性表的顺序表示

## ①顺序表的定义

| 存储地址            | 内存状态 | 位序  |
|-----------------|------|-----|
| $b$             | A1   | 1   |
| $b+s$           | A2   | 2   |
| ...             | ...  | ... |
| $b+(i-1)s$      | Ai   | i   |
| ...             | ...  | ... |
| $b+(n-1)s$      | An   | n   |
| $b+ns$          | 空    | 空   |
| ...             | ...  |     |
| $b+(maxlen-1)s$ | 空    |     |

顺序表中，逻辑上相邻的数据元素物理位置也是相邻的



## 2. 线性表的顺序表示

### ①顺序表的定义

| 存储地址            | 内存状态 | 位序  |
|-----------------|------|-----|
| $b$             | A1   | 1   |
| $b+s$           | A2   | 2   |
| ...             | ...  | ... |
| $b+(i-1)s$      | Ai   | i   |
| ...             | ...  | ... |
| $b+(n-1)s$      | An   | n   |
| $b+ns$          | 空    | 空   |
| ...             | ...  |     |
| $b+(maxlen-1)s$ | 空    |     |

计算顺序表中 $A_{i+1}$ 的地址

$$Array[i] = Array[0] + i * L$$

顺序表一般使用数组存储  
所以 $A_1$ 对应 $Array[0]$

顺序表中，逻辑上相邻的数据元素物理位置也是相邻的



# 2. 线性表的顺序表示

## ①顺序表的定义

| 存储地址            | 内存状态 | 位序  |
|-----------------|------|-----|
| $b$             | A1   | 1   |
| $b+s$           | A2   | 2   |
| ...             | ...  | ... |
| $b+(i-1)s$      | Ai   | i   |
| ...             | ...  | ... |
| $b+(n-1)s$      | An   | n   |
| $b+ns$          | 空    | 空   |
| ...             | ...  |     |
| $b+(maxlen-1)s$ | 空    |     |

计算顺序表中 $A_{i+1}$ 的地址  
 $Array[i] = Array[0] + i * L$

顺序表一般使用数组存储  
所以 $A_1$ 对应 $Array[0]$

|    |      |      |      |      |
|----|------|------|------|------|
| 数组 | A[0] | A[1] | A[2] | A[3] |
| 位序 | 1    | 2    | 3    | 4    |

顺序表中，逻辑上相邻的数据元素物理位置也是相邻的



# 2. 线性表的顺序表示

## ①顺序表的定义

| 存储地址            | 内存状态 | 位序  |
|-----------------|------|-----|
| $b$             | A1   | 1   |
| $b+s$           | A2   | 2   |
| ...             | ...  | ... |
| $b+(i-1)s$      | Ai   | i   |
| ...             | ...  | ... |
| $b+(n-1)s$      | An   | n   |
| $b+ns$          | 空    | 空   |
| ...             | ...  |     |
| $b+(maxlen-1)s$ | 空    |     |

计算顺序表中 $A_{i+1}$ 的地址  
 $Array[i] = Array[0] + i * L$

只要确定了存储的起始位置  
顺序表中任一数据元素都可**随机存取**

顺序表中，逻辑上相邻的数据元素物理位置也是相邻的



## 2. 线性表的顺序表示

### ①顺序表的定义

```
//动态
typedef struct{
    ElemType *elem; //存储空间的基地址
    int length; //当前长度
}SqList; //顺序表的结构类型为SqList
```

```
//静态
#define MaxSize 50 //定义线性表的最大长度
typedef struct{
    ElemType data[MaxSize]; //顺序表的元素
    int length; //当前长度
}SqList; //顺序表的结构类型为SqList
```

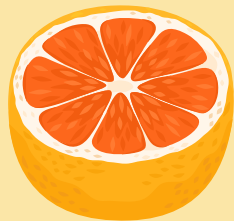


## 2. 线性表的顺序表示

### ①顺序表的定义

结构体的基本语法

```
struct Fruit{  
    ElemType Apple;  
    ElemType Banana;  
    ElemType Strawberry;  
}
```



```
typedef struct Fruit{  
    ElemType Apple;  
    ElemType Banana;  
    ElemType Strawberry;  
}Fru;
```

```
struct Fruit a; //申请果篮a  
sturct Fruit b; //申请果篮b
```

```
Fru a; //申请果篮a  
Fru b; //申请果篮b
```



## 2. 线性表的顺序表示

### ①顺序表的定义

```
//动态
typedef struct{
    ElemType *elem; //存储空间的基地址
    int length; //当前长度
}SqList; //顺序表的结构类型为SqList
```

```
//静态
#define MaxSize 50 //定义线性表的最大长度
typedef struct{
    ElemType data[MaxSize]; //顺序表的元素
    int length; //当前长度
}SqList; //顺序表的结构类型为SqList
```



## 2. 线性表的顺序表示

### ②顺序表的基本操作

初始化

```
//动态
Status InitList (SqList &L)
{
    L.elem = (ElemType*) malloc (sizeof (ElemType) *InitSize) ;
    //为顺序表分配一个大小为InitSize的数组空间
    if(!L.elem) exit(OVERFLOW); //存储分配失败退出
    L.length = 0; //空表长度为0
    return OK;
}
```



## 2. 线性表的顺序表示

### ②顺序表的基本操作

取值

```
//动态
Status GetElem (SqList L, int i, ElemType &e)
{
    if (i < 1 || i > L.Length) return error; //判断i的值是否合理
    e = L.elem[i-1]; //elem[i-1]单元存储第i个数据元素
    return OK;
}
```



## 2. 线性表的顺序表示

### ②顺序表的基本操作

按值查找

```
//动态
int LocateElem (SqList L, ElemType e) //在顺序表L中查找值为e的元素，返回其序号
{
    for (i = 0; i < L.length; i++)
        if(L.elem[i] == e)
            return i+1; //查找成功，返回序号
    return 0; //查找失败，返回0
}
```



## 2. 线性表的顺序表示

### ②顺序表的基本操作

插入

```
//动态
Status ListInsert (SqList &L, int i, ElemType e)
{
    if((i < 1) || (i > L.length + 1)) return error; //如果i值不合法, 返回error
    if(L.length == MAXSIZE) return error; //如果存储空间满, 返回error
    for(j = L.length - 1; j >= i-1; j--)
        L.elem[j+1] = L.elem[j]; //插入位置之后的元素后移
    L.elem[i-1] = e; //新元素e放入第i个位置
    L.length++; //表长加1
    return OK;
}
```



## 2. 线性表的顺序表示

### ②顺序表的基本操作

删除

```
//动态
Status ListDelete (SqList &L, int i)
{
    if((i < 1 || (i > L.length)) return error; //如果i值不合法, 返回error
    for(j = i; j <= L.length - 1; j++)
        L.elem[j-1] = L.elem[j]; //被删除元素之后的元素前移
    L.length--; //表长减1
    return OK;
}
```



## 真题训练

1. 下列对顺序存储的有序表(长度为 $n$ ) 实现给定操作的算法中, 平均时间复杂度为 $O(1)$ 的是

A. 查找包含指定值元素的算法

B. 插入包含指定值元素的算法

C. 删除第 $i$  ( $1 \leq i \leq n$ ) 个元素的算法

D. 获取第 $i$  ( $1 \leq i \leq n$ ) 个元素的算法



## 真题训练

1. 下列对顺序存储的有序表(长度为 $n$ ) 实现给定操作的算法中, 平均时间复杂度为 $O(1)$ 的是

A. 查找包含指定值元素的算法

B. 插入包含指定值元素的算法

C. 删除第 $i$  ( $1 \leq i \leq n$ ) 个元素的算法

D. 获取第 $i$  ( $1 \leq i \leq n$ ) 个元素的算法

**祝你今天特别特别开心！**